

Implementation of a Ray Tracer

Lab Report

Name: **Youssef Naji**

Course: **Computer Graphics**

Instructors: **Boneel Nicolas, Gong Xianjin**

Date: **May 2026**

1 Introduction

The objective of this project was to progressively implement a ray tracer over four laboratory sessions. The final renderer supports diffuse surfaces, mirror reflections, direct and indirect lighting, anti-aliasing, mesh rendering using the Möller–Trumbore algorithm, and acceleration using a Bounding Volume Hierarchy (BVH).

The code base initially contained a provided framework in `ex.cpp`, but the work of the labs mainly consisted in completing the `TODO` sections inside `main.cpp`. Therefore, this report focuses mostly on the algorithms and features that were directly implemented during the labs.

The final scene consists of a Cornell-box-like environment built from large spheres, together with a cat mesh loaded from an OBJ file. The renderer uses recursive path tracing with Monte Carlo sampling for indirect illumination.

The final rendering parameters were:

- Resolution: 1000×1000
- Samples per pixel: 32
- Maximum bounce depth: 20
- Gamma correction: 2.2
- Mesh: `cat.obj`

2 Lab 1: Basic Ray Tracing

The goal of the first lab was to implement the foundations of the renderer:

- Sphere intersection
- Recursive mirror reflections
- Direct illumination
- Hard shadows

2.1 Sphere Intersection

The first important step was implementing ray-sphere intersection. Given a ray:

$$R(t) = O + t\vec{u}$$

and a sphere centered at C with radius R , the intersection is obtained by solving:

$$\|O + t\vec{u} - C\|^2 = R^2$$

which leads to a quadratic equation.

The implemented code is:

```
double b = dot(ray.u, ray.O - C);
double delta = sqr(b) -
(dot(ray.O - C, ray.O - C) - R * R);

if (delta < 0) return false;

double t2 = -b - sqrt(delta);
double t1 = -b + sqrt(delta);
```

One important difficulty encountered during this part was selecting the correct solution among the two roots. Initially, intersections behind the camera or extremely close to the surface created rendering artifacts.

To avoid self-intersections, a small epsilon value was used:

```
if (t2 > 1e-6)
```

Without this epsilon, secondary rays often intersected the same surface again, producing black dots and shadow artifacts.

2.2 Closest Intersection

The scene intersection routine iterates through all objects and keeps only the nearest valid intersection:

```
if (objects[i] -> intersect(ray, Ptemp, ttemp, Ntemp)) {
    if (ttemp < t) {
        has_intersection = true;
        t = ttemp;
        P = Ptemp;
        N = Ntemp;
        object_id = i;
    }
}
```

At this stage the renderer already supported multiple spheres in the scene.

2.3 Direct Lighting and Shadows

Direct illumination was implemented using a point light source and Lambertian shading:

$$L = \frac{I}{4\pi r^2} \cdot \frac{\rho}{\pi} \cdot \max(0, N \cdot \omega)$$

where:

- I is the light intensity
- r is the distance to the light
- ρ is the surface albedo
- N is the normal

Shadows were implemented by casting a secondary ray toward the light source.

```
Ray shadow_ray(P + 1e-6 * N, light_direction);
```

A major issue during debugging was that some surfaces incorrectly shadowed themselves because the shadow ray started exactly on the surface. This again required adding a small offset along the normal.

2.4 Mirror Reflection

Mirror reflection was implemented recursively using:

$$R = I - 2(I \cdot N)N$$

with the code:

```
Vector reflected_direction =
ray.u - 2.0 * dot(ray.u, N) * N;
```

This was the first introduction to recursive ray tracing.

3 Lab 2: Indirect Lighting and Anti-Aliasing

The second lab introduced Monte Carlo path tracing.

The objective was to simulate indirect illumination by recursively sending random rays in the hemisphere around the surface normal.

3.1 Cosine-Weighted Sampling

The indirect rays were generated using cosine-weighted hemisphere sampling.

Two random numbers r_1 and r_2 are sampled uniformly:

```
double r1 = uniform(engine[tid]);
double r2 = uniform(engine[tid]);
```

Then spherical coordinates are generated:

```
double cos_t = sqrt(1.0 - r2);
double sin_t = sqrt(r2);
double phi = 2.0 * M_PI * r1;
```

The sampled direction becomes:

```
Vector indirect_dir =
sin_t * cos(phi) * u_dir +
sin_t * sin(phi) * v_dir +
cos_t * N;
```

Constructing the local orthonormal basis correctly was one of the main difficulties. In the beginning, several generated vectors were invalid because the chosen tangent vector became nearly parallel to the normal.

The solution implemented was to dynamically choose the least aligned axis before applying the cross product.

3.2 Recursive Indirect Illumination

Indirect illumination was recursively accumulated:

```
Ray indirect_ray(P + 1e-6 * N, indirect_dir);
Vector indirect = getColor(indirect_ray,
recursion_depth + 1);
```

The resulting color was multiplied by the surface albedo.

One important observation was that increasing the number of light bounces significantly improved realism but also drastically increased rendering time.

3.3 Anti-Aliasing

Anti-aliasing was implemented by randomly jittering rays inside each pixel.

```
double u = uniform(engine[thread_id]);
double v = uniform(engine[thread_id]);
```

The camera ray direction then becomes:

```

Vector ray_direction(
    j - (W / 2.0) + u,
    (H / 2.0) - i - v,
    -W / (2.0 * tan(scene.fov / 2.0))
);

```

Multiple samples per pixel were averaged:

```

for (int s = 0; s < SPP; s++) {
    color = color + scene.getColor(ray, 0);
}
color = color / SPP;

```

Without anti-aliasing, the edges of objects appeared extremely jagged. Increasing the number of samples reduced noise but increased computation time.

4 Lab 3: Mesh Rendering

The objective of the third lab was to support triangle meshes.

The cat model was loaded from an OBJ file:

```

cat.readOBJ("Models_F0202A090/cat.obj");

```

4.1 Möller–Trumbore Algorithm

Triangle intersection was implemented using the Möller–Trumbore algorithm.

The edges of the triangle are first computed:

```

Vector edge1 = B - A;
Vector edge2 = C - A;

```

The determinant test is:

```

Vector h = cross(ray.u, edge2);
double a = dot(edge1, h);

```

Barycentric coordinates are then evaluated:

```

double u = f * dot(s, h);
double v = f * dot(ray.u, q);

```

This algorithm was considerably harder to debug than sphere intersection because incorrect triangle orientation or barycentric tests immediately produced missing geometry.

Another issue was that triangle normals sometimes pointed in the wrong direction, producing incorrect lighting.

The implemented fix was:

```

if (dot(N, ray.u) > 0) {
    N = -1.0 * N;
}

```

4.2 Performance Problems

At this stage, rendering became extremely slow because every ray had to test intersections against all triangles of the mesh.

Even though the cat model is relatively small, recursive indirect lighting generated a very large number of rays.

This motivated the implementation of acceleration structures in the next lab.

5 Lab 4: BVH Acceleration

The final lab introduced Bounding Volume Hierarchies (BVH).

The purpose of the BVH is to avoid testing every ray against every triangle.

5.1 Bounding Boxes

Each BVH node stores an axis-aligned bounding box:

```
Vector bbox_min;  
Vector bbox_max;
```

Before testing triangles, the renderer first checks whether the ray intersects the box.

```
if (!bbox_intersect(ray,  
    node.bbox_min,  
    node.bbox_max, t)) {  
    return false;  
}
```

5.2 Recursive BVH Construction

The BVH was recursively constructed by splitting triangles according to their centroid coordinates.

```
std::nth_element(indices.begin() + start,  
    indices.begin() + mid,  
    indices.begin() + end,  
    ...  
);
```

The splitting axis was chosen according to the largest extent of the centroid bounding box.

5.3 Performance Improvement

The BVH produced a dramatic improvement in rendering time.

Without BVH, rendering the mesh became nearly unusable when indirect lighting and multiple bounces were enabled.

With BVH acceleration, only a small subset of triangles needed to be tested for each ray.

This was probably the most important optimization of the whole project.

6 Final Results

The final renderer supports:

- Diffuse surfaces
- Mirror reflections
- Hard shadows
- Recursive indirect lighting
- Anti-aliasing
- Triangle mesh rendering

- BVH acceleration

The final rendered scene contains:

- A Cornell-box-style environment
- A cat OBJ mesh
- Recursive global illumination

7 Conclusion

This project provided a practical introduction to physically-based rendering and Monte Carlo path tracing.

The most difficult parts were:

- Debugging numerical precision issues
- Implementing recursive indirect lighting
- Correctly constructing orthonormal bases
- Optimizing mesh rendering using BVH

The project also demonstrated the tradeoff between realism and computation time. Increasing the number of samples per pixel or the recursion depth significantly improved visual quality, but also increased rendering cost.

Overall, the project gave a much better understanding of how modern rendering engines and physically-based renderers work internally.